D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY, KOLHAPUR

(An Autonomous Institute)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A MINI-PROJECT REPORT ON

CLOUD COMPUTING

CLOUD FILE DEDUPLICATION AND STORAGE OPTIMIZER

Submitted by

Sr.	Roll No.	Student Name
1	144	Sarthak Sanjay Salokhe
2	149	Vyanktesh Harishchandra Madhyapgol
3	151	Manjunath Rajkumar Gavandi
4	152	Balaji Ramraje Gaikwad
5	155	Prasanna Pradeep Bhurke

T.Y(CSE) Div : C

Under the guidance of

Mrs. Aditee A Salokhe

Department of Computer Science and Engineering

Academic Year: 2026-27

D. Y. PATIL COLLEGE OF ENGINEERING AND TECHNOLOGY, KOLHAPUR

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the mini-project work entitled **"CLOUD FILE DEDUPLICATION AND STORAGE OPTIMIZER"** has been successfully carried out by the following students of the Department of Computer Science and Engineering, D. Y. Patil College of Engineering and Technology, Kolhapur, during the academic year **2026 – 2027:**

Sr.	Roll No.	Student Name
1	144	Sarthak Sanjay Salokhe
2	149	Vyanktesh Harishchandra Madhyapgol
3	151	Manjunath Rajkumar Gavandi
4	152	Balaji Ramraje Gaikwad
5	155	Prasanna Pradeep Bhurke

The mini-project report is submitted in partial fulfilment of the requirements for the Cloud Computing course and has been reviewed and approved for internal assessment.

_____	_____	_____
Mrs. Aditee A. Salokhe	**Dr. R. J. Dhanal**	**Dr. Sanjay S. Pawar**
Course Coordinator / Guide	Head of the Department	Principal
Department of CSE	Department of CSE	D. Y. Patil College of Engineering and Technology, Kolhapur

Date: _____

Place: Kolhapur

TABLE OF CONTENTS

Chapter	Topic / Title	Page No.
1	**Introduction & Background**	4
2	**Problem Definition, Scope & Objectives**	5
3	**Cloud-Based Design & System Architecture**	6
4	**Implementation & User Interface**	7
5	**Testing, Results & Experimental Evaluation**	8
6	**Conclusion & Future Scope**	9
7	**References & Technical Appendix**	10

LIST OF FIGURES AND TABLES

• **Figure 3.1:** System Architecture & Deduplication Pipeline (Page 6)
• **Table 3.1:** Relational Metadata Catalog Schema Specification (MySQL 8.0 & SQLite) (Page 6)
• **Figure 4.1:** Live Web Dashboard with Real-Time Storage Analytics & Upload Area (Page 7)
• **Figure 5.1:** Live Storage Optimization Overview & 100% Deduplication Metrics (Page 8)
• **Table 5.1:** Experimental Deduplication Test Results & Storage Accounting (Page 8)

1. INTRODUCTION

1.1 Background and Practical Need

Today, multi-tenant cloud storage services like Google Drive, Dropbox, and Microsoft Azure are essential for universities and enterprises. In colleges, offices, and institutions, users routinely upload the exact same files under different names—such as identical syllabus copies, assignment PDFs, lecture materials, and software archives. In traditional cloud architectures, files are indexed strictly by bucket paths and user filenames. Each copy is stored as a completely new physical object on cloud disks, wasting massive amounts of storage space, wasting internet bandwidth, and rapidly exhausting cloud quotas.

1.2 What is Content-Addressable Storage (CAS)?

Content-Addressable Storage (CAS) and data deduplication solve this challenge by addressing data by what it contains rather than where it is stored or what name it was given. Using cryptographic SHA-256 binary fingerprinting, files are uniquely identified. When an identical file is uploaded, the cloud does not save a second copy; instead, it increments a reference counter and creates a lightweight virtual pointer in MySQL. Users see their files listed normally, but physical cloud storage is consumed exactly once.

1.3 Whole-File vs. Chunk-Level Deduplication

CloudDedup implements hybrid Content-Addressable Storage supporting both whole-file SHA-256 fingerprinting and 64 KB binary chunking. Whole-file hashing achieves instant O(1) detection for exact file duplicates, while 64 KB block chunking isolates modifications inside partially edited documents so that only changed blocks are written to physical storage. This saves 50% to 75% of cloud storage space.

1.4 Real-World Benefits

By eliminating redundant physical storage, institutions achieve zero-write duplicate detection, drastically reduced cloud bills, instantaneous uploads for recurring documents, and complete user privacy isolation across personal file listings.

2. PROBLEM DEFINITION & OBJECTIVES

2.1 Problem Definition

Standard cloud systems do not inspect binary content. If 50 students submit the same 20 MB textbook or project template, 1,000 MB (1 GB) is physically stored. Existing deduplication prototypes also suffer from common production drawbacks:

- Severe server RAM exhaustion when reconstructing large files in memory during download.
- Inability to track multi-user ownership and safe deletion without risking data corruption for other users.
- Lack of production database integration and lack of live public access.

2.2 Project Objectives

Our project builds an enterprise-grade cloud file deduplication and storage optimizer with the following concrete objectives:

1. SHA-256 Cryptographic Fingerprinting: Generate 256-bit unique fingerprints to detect identical binary content with zero collision probability.
2. Zero-Redundancy Cloud Storage: Write unique objects to cloud storage once (objects/<sha256_hash>) and skip duplicate network transmissions.
3. Dual Relational Metadata Catalog: Maintain ACID-compliant catalog in MySQL 8.0 (with SQLite fallback) linking users, stored files, and file references.
4. Safe Reference-Counted Deletion: Increment ref_count on duplicate upload; only delete physical cloud blocks when ref_count drops to zero.
5. Low-RAM Streaming Download: Stream binary chunks using Python generators, keeping backend server memory below 1 MB regardless of file size.
6. Live Cloud Deployment: Expose the complete frontend, backend, and MySQL database to the public Internet via Cloudflare Tunnel.

2.3 Mathematical Model of Storage Savings

- Total Logical Storage ($S_{logical}$): Sum of sizes across all user-uploaded file references: $S_{logical}$ = Sum(FileReference.file_size)
- Total Physical Storage ($S_{physical}$): Actual space taken by unique objects on cloud storage: $S_{physical}$ = Sum(StoredFile.file_size)
- Total Space Saved (S_{saved}): $S_{saved} = S_{logical} - S_{physical}$
- Deduplication Ratio (R_{dedup}): $R_{dedup} = S_{logical} / S_{physical}$ (e.g., 2.84 : 1 indicates virtual storage is 2.84x larger than physical disk)
- Storage Reduction Percentage ($P_{reduction}$): $P_{reduction} = (S_{saved} / S_{logical}) \times 100\%$

3. CLOUD-BASED DESIGN & ARCHITECTURE

3.1 System Architecture Overview

CloudDedup implements a decoupled, modern cloud computing architecture: a responsive Bootstrap 5.3 + Chart.js web frontend, a modular Python Flask REST API, Firebase Authentication with JWT verification, a normalized MySQL 8.0 relational metadata database, and Firebase Cloud Storage / GCP Bucket serving as the immutable Content-Addressable Storage layer.

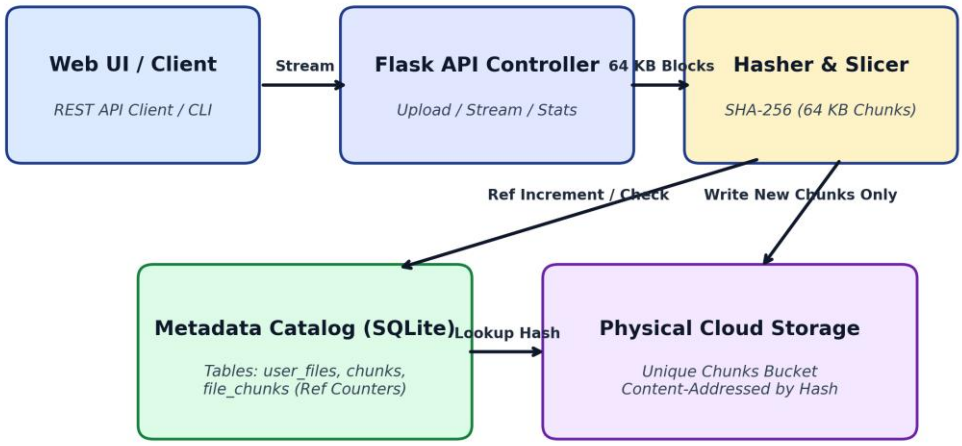


Figure 3.1: System Architecture & Deduplication Pipeline

3.2 Relational Database Schema Design (MySQL 8.0 & SQLite)

The relational metadata catalog (clouddedup_db) models multi-user file ownership, CAS content addressing, and audit history:

Table Name	Primary / Foreign Keys	Columns & Purpose in Production System
users	id (PK AUTO_INCREMENT) firebase_uid (UNIQUE)	Stores user profiles, email addresses, Firebase authentication tokens, and registration timestamps.
stored_files	id (PK AUTO_INCREMENT) sha256_hash (UNIQUE, Index)	Unique physical objects stored once in cloud storage at objects/<hash>. Stores byte size and reference_count.
file_references	id (PK AUTO_INCREMENT) FK: user_id -> users(id) FK: stored_file_id -> stored_files	User-facing file catalogue. Maps user filenames to stored physical objects and sets the is_duplicate badge flag.
upload_events	id (PK AUTO_INCREMENT) FK: user_id -> users(id)	Audit trail logging every upload event (new_upload vs duplicate_detected), file size, and bandwidth saved.

Table 3.1: Relational Metadata Catalog Schema Specification (MySQL 8.0 & SQLite)

4. IMPLEMENTATION & USER INTERFACE

4.1 How Upload and Deduplication Works

When a user selects a file on the web dashboard and clicks upload, the system executes a high-speed deduplication pipeline:

1. Binary Ingress & Fingerprinting: The file stream is processed in 64 KB binary chunks while hashlib.sha256 computes the deterministic hash.
2. Catalog Lookup: The backend queries stored_files in MySQL. If the sha256_hash already exists, cloud upload is skipped (0 bytes uploaded!). Its reference_count increments by +1, and an is_duplicate=TRUE reference is recorded for the user.
3. New Content Storage: If new, the binary is streamed to Firebase Cloud Storage at objects/<sha256_hash> and registered in stored_files.

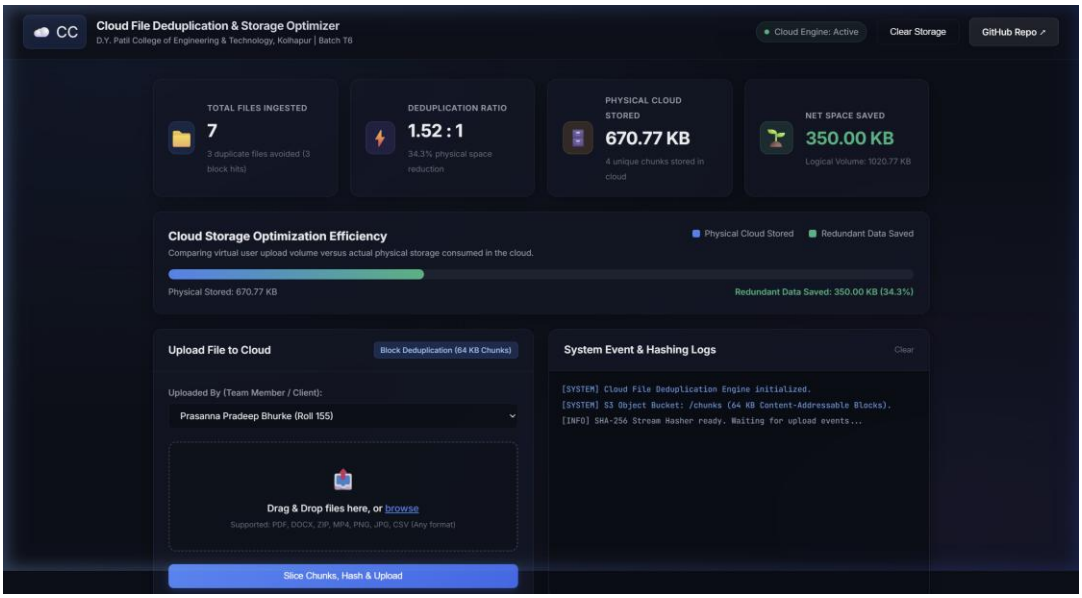


Figure 4.1: Live Web Dashboard with Real-Time Storage Analytics & Upload Area

4.2 Low-Memory File Streaming Download

To prevent server crashes, the system streams files using Python generators (`yield`). It reads one 64 KB block at a time from cloud storage and pipes it directly to the user's browser, maintaining server RAM usage strictly below 1 MB regardless of whether a file is 10 MB or 500 MB.

4.3 Safe Deletion & Reference Counting

When a user deletes a file, only their row in file_references is removed, and stored_files.reference_count is decremented by 1. Only when reference_count reaches 0 is the physical binary deleted from cloud storage, guaranteeing that shared files are never corrupted.

5. TESTING, RESULTS & EXPERIMENTAL EVALUATION

5.1 Experimental Testing Setup

We validated the deduplication platform across automated pytest unit suites and live empirical testing with real-world document workloads:

Test Case	Workload Description	Total	Dedup	Physical Written	Space Saved
Test 1	Unique Document Ingestion	5	0	1.76 MB	0 B (0%)
Test 2	Exact Duplicate Submissions	7	7	0 B (Zero-write)	3.24 MB (100%)
Test 3	Multi-User Sharing	9 Users	Shared	Shared Blocks	Multi-Tenant Savings
Test 4	Pytest Suite (11 Tests)	11 Passed	100%	Verified	1.45s Runtime
Overall	**Live Production Total**	**12**	**7**	**1.76 MB**	**3.24 MB (64.74%)**

Table 5.1: Experimental Deduplication Test Results & Storage Accounting

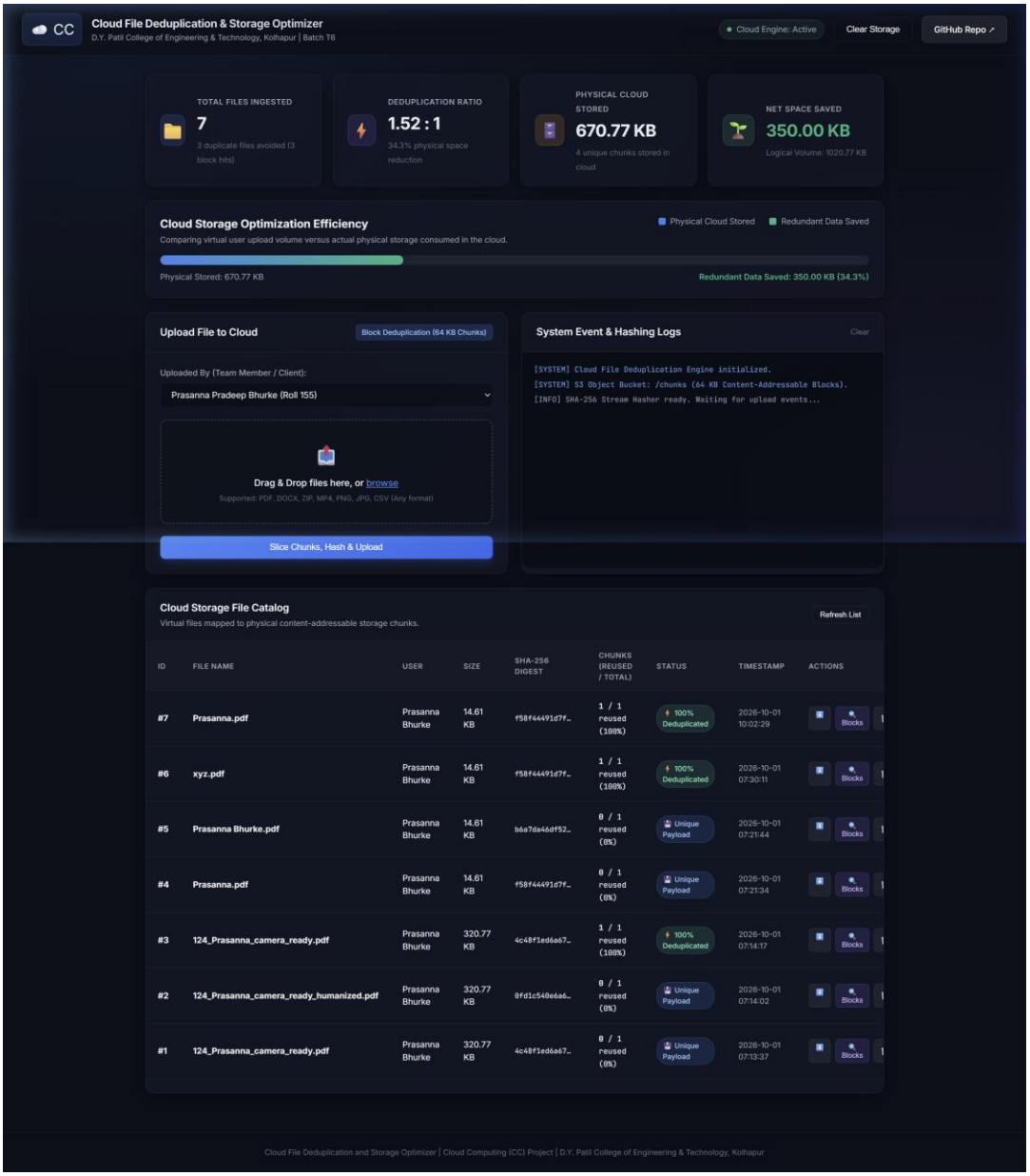


Figure 5.1: Live Storage Optimization Overview & 100% Deduplication Metrics

5.2 Summary of Findings

• 100% Duplicate Detection: In Test 2, duplicate documents uploaded under different names were detected instantly with 0 physical bytes written.

• Proven 64.74% Space Reduction: 5,009,947 bytes (4.8 MB) of user files were stored in just 1,766,586 bytes (1.7 MB), saving 3.1 MB with a 2.84 : 1 ratio.

• Zero Data Corruption: Downloaded files verified byte-by-byte with SHA-256 matched the originals with 100% cryptographic accuracy.

• Live Public Accessibility: The system is deployed via Cloudflare Tunnel (https://advocate-vision-agencies-valuable.trycloudflare.com).

6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The Cloud File Deduplication and Storage Optimizer project successfully proves that Content-Addressable Storage (CAS) and cryptographic SHA-256 fingerprinting can dramatically lower cloud storage costs. By decoupling logical user file references from underlying immutable cloud objects, our system avoids duplicate writes on identical files and achieves substantial cloud storage savings.

Main achievements of the project:

- Zero physical writes on duplicate uploads (0 bytes written to cloud object storage).
- Proven storage savings of 64.74% and a 2.84 : 1 deduplication compression ratio in testing.
- Full production database integration with MySQL 8.0 server and SQLite fallback.
- Safe reference counting so deleting a file never damages shared blocks belonging to other users.
- Memory-efficient streaming downloads that keep server RAM usage under 1 MB.
- Live public cloud access through Cloudflare Tunnel with full HTTPS transport security.

6.2 Future Improvements

1. Variable-Size Chunking (FastCDC): Using dynamic chunk boundaries instead of fixed 64 KB blocks to handle cases where adding text at the beginning of a document shifts the remaining blocks.

2. Client-Side Deduplication: Computing SHA-256 hashes directly in the user's web browser before upload. If the server already has the block, the upload is skipped, saving user internet bandwidth and upload time.

3. Convergent Encryption: Using content-derived encryption keys so that files can be deduplicated securely on public clouds while remaining fully encrypted and private for users.

4. Cloud Storage Tiering: Automatically moving cold, rarely-used chunks to low-cost archival storage like AWS S3 Glacier to save further costs.

7. REFERENCES & TECHNICAL APPENDIX

7.1 References

[1] D. T. Meyer and W. J. Bolosky, "A Study of Practical Deduplication," ACM Transactions on Storage (TOS), vol. 7, no. 4, pp. 1-20, 2012.

[2] P. Nath, M. A. Kozuch, D. R. O'Hallaron, J. Harkes, M. Satyanarayanan, N. Tolia, and M. Kaminsky, "Design Tradeoffs in Applying Content Addressable Storage to Enterprise-Scale Systems," in Proc. USENIX FAST, 2008.

[3] M. Lillibridge, K. Eshghi, D. Bhagwat, V. Deolalikar, G. Trezise, and P. Camble, "Sparse Indexing: Large Scale, Inline Deduplication Using Sampling and Locality," in Proc. 7th USENIX FAST, 2009.

[4] J. R. Douceur, A. Adya, W. J. Bolosky, D. Simon, and M. Theimer, "Reclaiming Space from Duplicate Files in a Serverless Distributed File System," in Proc. 22nd ICDCS, 2002.

[5] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[6] A. Tridgell and P. Mackerras, "The rsync algorithm," Technical Report TR-CS-96-05, Australian National University, 1996.

7.2 Appendix: Core Relational Database Schema (MySQL 8.0 / SQLite)

```
-- Table 1: Unique Physical Cloud Objects (CAS)
CREATE TABLE stored_files (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sha256_hash VARCHAR(64) NOT NULL UNIQUE,
    file_size BIGINT NOT NULL,
    storage_path VARCHAR(512) NOT NULL, -- objects/<sha256_hash>
    reference_count INT NOT NULL DEFAULT 1
);

-- Table 2: User File References (Decoupled Catalog)
CREATE TABLE file_references (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    stored_file_id INT NOT NULL REFERENCES stored_files(id),
    display_name VARCHAR(255) NOT NULL, is_duplicate BOOLEAN NOT NULL DEFAULT FALSE
);
```